API Systems - Task 1

Nirvana Vella IDM 5.2

Brief Description of Application

This application will serve as an infotainment for Maltese Citizens. This application will be focusing around an **interactive map** where a user will be able to **zoom around** the map to find **interactive elements like bubbles** which will indicate there is news in the locality. The target audience will be **25 and under specifically for students or those interested in journalism.** Journalists will be able to post on this application under a Journalism Membership. This application will also allow users to comment under articles, bookmark certain articles as well. Similar to social media but with verification systems and actual credibility. There will also be tags on top of each article and image where a verifier must have verified the articles/images. **[Not Verified -> In Progress -> Verified]**

API Implementation

This application will be similar to many social medias such as Instagram and X, where instead of everyone posting whatever they please it will be purely for journalistic purposes and posting will be limited to the user types.
This application will have an API that will fetch users' data, the articles spread throughout the map and their data. Users' data can be updated as well as their membership types if they purchase a certain membership.
The following will be the an API with these core resources within it;

- Users Resource
  - This will get the user's information, allow profile editing, enable the different dashboards that users will have access to *(User, Journalistic User & Verifier/Admin User),* and external authentication using Oauth 2.0 which is mentioned later on to allow users to link their accounts with other applications *(Facebook, Instagram or X).*
- Articles Resource
  - This will be dependent on the type of user, if they are a regular user they will not be allowed to post so they will only have GET endpoints for this type of user. Then the Journalistic and the Admin users will be allowed to POST but only the Admin will be able to **verify** the posts done by the Journalistic users. The articles resource would also GET comments from all the users under specific articles if there were any. Once an article is posted it will immediately be set as Unverified and then follow the process: **Unverified** (no verifiers has seen it yet) -> **In Progress** (A verifier is actively looking whether it's real or fake) -> **Verified**
- Map Location Resource
  - This resource will get the user's location and as well as the articles locations around the map which will be centered around Malta primarily. Similarly to

Snapchat where an Avatar is shown on a map but instead an interactive element will be there depending on the number of articles.

- Bookmarking Resource
  - This resource will track which articles have been bookmarked by the user themselves, meaning it will get the articles and also it will update and add the articles items within a list.
- Membership Resource
  - This resource will track how long the membership has left, the membership type whether its activated or not, it will update once the user has accepted the membership and it will be deactivated if the user decides to cancel the membership. Processes any payments necessary and verifications using PATCH method and as well as a payment gateway.

RESTful APIs

What are APIs?

API stands for Application Programming Interface, this allows different systems or devices to communicate together. With API it avoids creating redundant code and/or scripts and only handles backend code meaning no user interface or user experience is involved here.

What is REST API?

RESTful APIs comes from a type of API called **REST** which stands for *'Representational State Transfer'.* REST APIs are typically used for web services due to their dynamic and versatile development; this type of API defines how applications communicate using HTTP methods. HTTP methods use specific operations that tell the resource what to do with said resource. The primary operations used by HTTP are the following:

HTTP Methods:

GET	Retrieves resources
POST	Creating new resources, more specifically a new record
PUT	Replaces resources entirely
PATCH	Partial updates (e.g. changing a password)
DELETE	Removes a resource entirely

REST API is ideal for public applications that require a broad compatibility, applications that use CRUD structure, mobile applications that require flexible data formats and internal APIs that connect microservices. A good example of REST API is *Spotify Streaming website.*
The HTTP method commonly creates, reads, updates or deletes (**CRUD**) resources. Resources are pieces of data meaning most things i.e. users, orders, products within a web which are essential to create a responsive and dynamic website.

Request/Response Process:

When the client makes a request within a REST API architecture the following steps happen

1. HTTP request with methods, headers, and optional body gets sent by client.
2. The server receives and processes the request.
3. The server returns the HTTP responses with the status code (e.g. 200 = successful response) and data.
4. Client receives the data retrieved and interprets the response.

Basic Response Codes:

200	Successful Request
201	Request Successfully Created
400	Bad Request: Invalid Client Request
404	Not Found: Resources does not exist
500	Internal Server Error: Server Failure

Core Principles of REST APIs

Client-server

The client-server design pattern enforces separation of concerns, which helps the client and the server components to evolve independently. By separating the client aka the user interface (UI) concerns from the server aka the data stored, portability of the user interface is improved across multiple platforms as well as improvement of scalability with simplification of server components.

Stateless

Due to its stateless nature, each request must contain all the necessary information as when the server's session ends all the data will not be kept, making it volatile. Each request from the client must contain all the information as previously mentioned for the server to understand and

complete the request sent. *E.g. Login information will not be stored so the information must be typed correctly or else the request will not be accepted.*

Uniform Interface

There are four constraints that allow developers to achieve a uniform REST interface; **Identification of resources, manipulation of resources, self-descriptive messages, hypermedia as the engine of application state.** Meaning, REST API follows consistent and uniform interface for interactions between the client and the servers i.e. HTTP standard methods that utilise the following *'GET, POST, PUT, DELETE '* and many more.

The four constrains:

Identification of Resources:

- Each resource must be uniquely identified involved within interactions between the client and the server by the interface.

Manipulation of resources through representations:

- The resources should have uniform representations in the server response as the API consumers should have access to modify the resources state within the server.

Self Descriptive Messages:

- Each resource must carry enough information to process the message as well as providing information on additional actions that the client can perform with that resource.

Hypermedia as the Engine of Application State:

- The client only has access to the initial URI of the application, enhancing its dynamicness when the client application drives other resources and interactions with the use of hyperlinks.
    - **URI:** Uniform Resource Identifier which are similar to URLs (Uniform Resource Locator). **URI** identifies what the resource is where **URL** identifies where the resource is.

Cacheable:

- Cache requires responses that implicitly/explicitly label themselves as cacheable or non-cachable. If the responses are cacheable, the client app is able to reuse the response data later on for equivalent requests and specified periods.
    - Cache is the temporary storage that keeps copies of specified data to reduce redundancy of requests allowing the web to be faster.
    - In REST APIs, web-based cache is used as REST constraints specifically refer to HTTP cacheability; the ability of clients and intermediaries to cache responses.
    - Storing **GET** responses can allow developers to improve performance with standard automation and caching responses.

REST APIs vs RESTful APIs

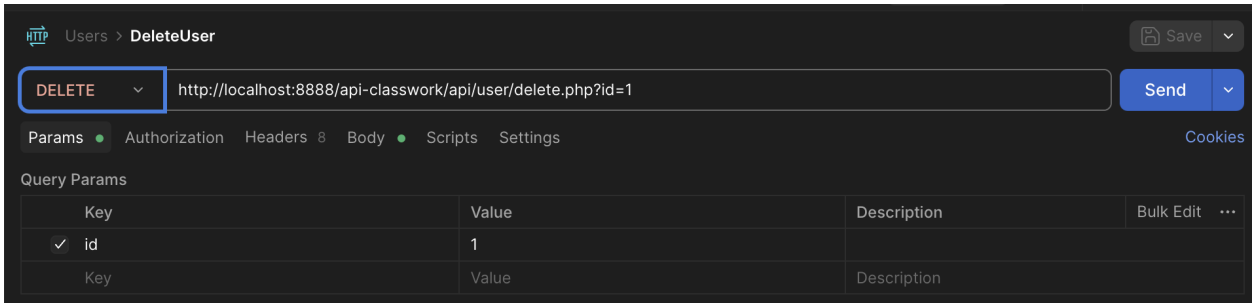
The main difference between REST and RESTful APIs is scalability and standardization. RESTful APIs are built from REST API's strict adherence to REST principles creating a more standardized approach to their API design.

REST API	RESTful API
Flexible and adaptable	Standardization for complex and large systems
Easy and adaptable integrations	Scalable solutions provided for business data systems
Statelessness and integration challenges	Detailed planning and strict principle adherence

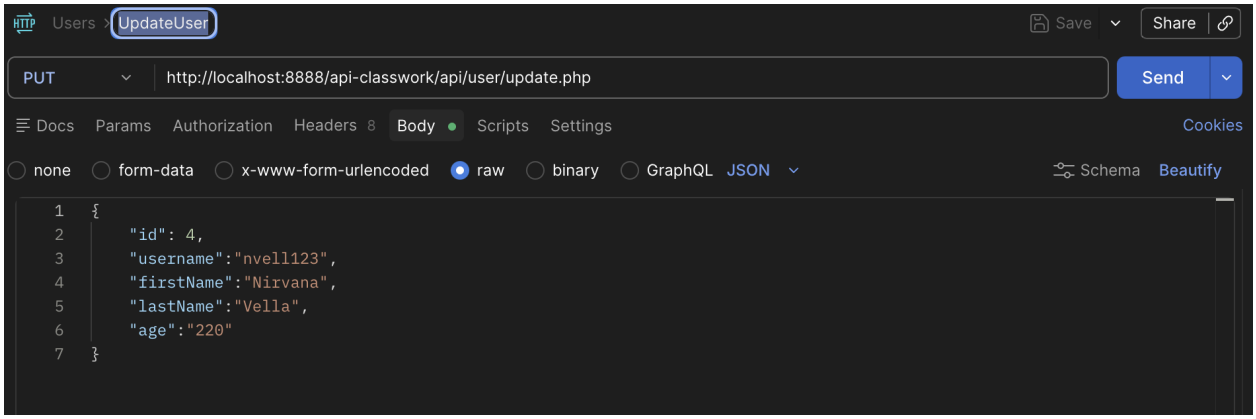
QueryString Parameters

Query parameters are used to pass information to an API in a simplified and flexible method, these are added to the end of the API endpoint URL as a series of key-value pairs.
E.g. https://www.example.com/api/items?**sort**=asc&**section**=sweets
In the example above 'sort' and 'section' are the query parameters that are passed through the API to specify whether it is in ascending or descending order in this case its ascending and the section of items to retrieve for the method.
Meaning QueryString Parameters within a REST API will primarily handle, filtering, sorting, pagination, searching, formatting and many more.
The most frequent cases where query strings are utilised is for filtering even more specifically pagination and searching.

An Example of QueryString Parameters in PostMan:



Request Body Example in Postman:



QueryString Syntax

?	Is a required separator between a URL and the parameters within a querystring.
&	Is similar to '?' but is a separator for multiple parameters.
=	Is what assigns value to parameter keys.
URL Encoding	Consists of encoding special characters such as, spaces becoming %20, and '&' becomes %26.

Headers

Headers are found in every HTTP request where headers contain additional information about the request such as the format of request body, preferred language, authentication, and many more. Headers are key-value pairs, the key is a string describing the type of information provided and the value is the actual data. The most common headers used in REST APIs are the following; **'Content-Type', 'Accept', 'Authorization', and 'User-Agent'.** Headers also provide additional information to control the behaviour of an API.

Endpoints for Assignment

Method	Endpoint	Description	Source Codes	QueryStrings (If any)	Request Body
GET	/api/users	Getting the specific user	200 - Success 404 - Account Not Found	n/a	n/a

GET	/api/users	Getting the user's location	200 - Success
			401 - Unauthorized
				n/a	{
					location
					}

POST	/api/users	Creating a new user account	200 - Success	n/a	n/a

PATCH	/api/users/{id}/password	Changing the password if the user is logged in but wants to change	204 - Successfully Updated
			401 - Unauthorized
				n/a	{
					currentPass, newPass
					}

GET	/api/articles	Getting all the articles	200 - Success	n/a	n/a

POST	/api/articles	Creating and uploading an article (Only users with a Journalistic Membership)	200 - Success	n/a	{
					title, content, location
					}

POST	/api/bookmarks	Adding an article to the bookmarks	201 - Successfully Created	n/a	{
					articleId
					}

DELETE	/api/bookmarks	Removing the article from the bookmarks	200 - Success 404 - Resource does not exist	n/a	n/a
GET	/api/users	Getting the user's membership type (even regular users will have a membership type)	200 - Success 404 - Resource does not exist	n/a	n/a

PATCH	/api/users	Updating a user's membership type only	204 - Successfully Updated
			401 - Unauthorized
				n/a	{
					membershipId
					}

Authorization

OAuth 2.0 is a type of authorization that stands for ***"Open Authorization",*** was a standard designed specifically to allow websites and/or applications to access resources hosted by other web applications. It is now the industry standard for online authorization after replacing OAuth 1.0 over a decade ago (2012). OAuth 2.0 provides the consented access and restrictions of actions on what the client app is allowed to perform with the resources allowed on behalf of the user.

OAuth 2.0 is strictly an authorization protocol and has no authentication features, as it was primarily designed to grant access to the resources necessary like remote APIs and user data. This protocol uses Access Token, meaning a piece of data which represents authorization to the accessed resources. In this open authorization the **JSON web token (JWT)** format is most commonly used.
Scopes are used to define specific actions and limitations within an application preventing access to a user's entire data. Scopes are used when a third-party application requests access by sending a parameter to the authorization server listing all the required permissions. It also ties in with user consent of what they allow to be accessed specifically data such as sharing their email, or their activity. The resource server also checks when a scope is sent that it is returned in the access token.

OAuth 2.0 Flow

Before OAuth 2.0 can be accessed or used, the client must get their own credentials which are the following: **Client Id and Client Secret.** These are acquired from the Authorization Server that is implemented within a system to identify and actually authenticate itself when an Access token is requested.

The Client Id and Client Secret are unique credentials, while the Client Id is public, the client secret is similar to a password which **must be confidential** as a security precaution as without it users can impersonate other users, and securing access means protecting user data.
<u>Actual Flow of OAuth 2.0:</u>

1. The client requests authorization from the Authorization Server, providing its credentials, the desired scopes, and the redirect endpoint to receive the Access Token or Authorization Code.
2. The Authorization Server authenticates the client and verifies the requested scopes against what is permitted.
3. The resource owner interacts with the Authorization Server to grant or deny access.
4. The Authorization Server then redirects back to the client with an Authorization Code or Access Token, depending on the grant type. A Refresh Token may also be issued.

   Authorization: Bearer <access_token>

5. Using the Access Token, the client requests access to the protected resource from the Resource Server.

Implementation within Application

With the implementation of OAuth 2.0 and its scopes, this will strengthen security especially when involving one of the biggest security threats one being, **Broken Function Level Authorization** which will be explained further on**.**

Roles and their Scopes:

Regular User -> read:articles, comment, bookmarks
Journalist -> read:articles, write:articles, edit:articles
Verifier -> verify:articles, delete:articles, manage:users

Some Endpoints with Previously Mentioned Scopes

Endpoint	Method	Scope
/articles	GET	read:articles
/articles	POST	write:articles
/articles/{id}/verify	PATCH	verify:articles
/user/bookmarks	GET	bookmarks

Security Considerations

OWASP is an industry standard document specifically for developers and web applications security, which covers a broad variety of the most frequent or critical security issues. This is updated frequently to help developers mitigate any vulnerabilities for a more secure application.

Top 10 API Security OWASP (2023)

Official Source: https://owasp.org/www-project-api-security/

1. **Broken Object Level Authorization**
    a. APIs expose endpoints that handle object identifiers, creating a wide attack surface of Object Level Access Control issues. Object level authorization checks should be considered in every function that accesses a data source using an ID from the user.
2. **Broken Authentication**
    a. Authentication mechanisms are often implemented incorrectly, allowing attackers to compromise authentication tokens or to exploit implementation flaws to assume other user's identities temporarily or permanently. Compromising a system's ability to identify the client/user, compromises API security overall.
3. **Broken Object Property Level Authorization**
    a. This category combines Excessive Data Exposure and Mass Assignment, focusing on the root cause: the lack of or improper authorization validation at the object property level. This leads to information exposure or manipulation by unauthorized parties.
4. **Unrestricted Resource Consumption**

    a. Satisfying API requests requires resources such as network bandwidth, CPU, memory, and storage. Other resources such as emails/SMS/phone calls or biometrics validation are made available by service providers via API integrations, and paid for per request. Successful attacks can lead to Denial of Service or an increase of operational costs.
5. **Broken Function Level Authorization**
    a. Complex access control policies with different hierarchies, groups, and roles, and an unclear separation between administrative and regular functions, tend to lead to authorization flaws. By exploiting these issues, attackers can gain access to other users' resources and/or administrative functions.
6. **Unrestricted Access to Sensitive Business Flows**
    a. APIs vulnerable to this risk expose a business flow - such as buying a ticket, or posting a comment - without compensating for how the functionality could harm the business if used excessively in an automated manner. This doesn't necessarily come from implementation bugs.
7. **Server Side Request Forgery**
    a. Server-Side Request Forgery (SSRF) flaws can occur when an API is fetching a remote resource without validating the user-supplied URI. This enables an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall or a VPN.
8. **Security Misconfiguration**
    a. APIs and the systems supporting them typically contain complex configurations, meant to make the APIs more customizable. Software and DevOps engineers can miss these configurations, or don't follow security best practices when it comes to configuration, opening the door for different types of attacks.
9. **Improper Inventory Management**
    a. APIs tend to expose more endpoints than traditional web applications, making proper and updated documentation highly important. A proper inventory of hosts and deployed API versions also are important to mitigate issues such as deprecated API versions and exposed debug endpoints.
10. **Unsafe Consumption of APIs**
    a. Developers tend to trust data received from third-party APIs more than user input, and so tend to adopt weaker security standards. In order to compromise APIs, attackers go after integrated third-party services instead of trying to compromise the target API directly.

2 OWASP Implementations for Threat Mitigation

The two most vital for this type of application would be Broken Object Level Authorization (API no. 1) and Broken Function Level Authorization (API no. 5).

Broken Object Level Authorization (API no. 1) Within Application

Due to this app needing to expose endpoints such as userIds, articleIds and many more endpoints, this can lead to users being able to easily manipulate the object IDs if sequential IDs

are used (userId = 1, userId = 2… etc.), allowing users to easily guess the IDs and gaining access.

How it will be Mitigated:

1. Authorization checks that use for every function that runs or accesses resources by ID.
2. Randomizing the ID numbers to ensure uniqueness is involved making the IDs unpredictable by utilising an **Universally Unique Identifier (UUID)**, which is a 36-character alphanumeric string that can be used to identify information.

Broken Function Level Authorization (API no. 5) Within Application

Due to the application having three different users **(Regular User, Verifier, & Journalistic)** with each their own set of allowed functionalities *e.g. a regular user will not be allowed to post an article while a Journalistic User is allowed.*

How it will be Mitigated:

1. Enforcing role check at every endpoint by using OAuth 2.0 using scopes, to navigate the different permissions across each role.

Reference list

Burgess, R. (2023) *Using query parameters and headers in REST API design*, *Lonti.com*. Toro Cloud Pty Ltd. Available at: https://www.lonti.com/blog/using-query-parameters-and-headers-in-rest-api-design#:~:text=Query%20parameters%20can%20be%20used%20to%20control%20the%20number%20of,results%20from%20the%2020th%20item..

Custer, C. (2023) *What is a UUID, and why should you care?*, *Cockroach Labs*. Available at: https://www.cockroachlabs.com/blog/what-is-a-uuid/.

Gupta, L. (2019) *What is REST – Learn to create timeless REST APIs*, *Restfulapi.net*. Available at: https://restfulapi.net/.

OWASP (2019) *OWASP API Security - Top 10 | OWASP*, *owasp.org*. Available at: https://owasp.org/www-project-api-security/.

Ploesser, K. (2019) *REST API Design Best Practices for Parameter and Query String Usage*, *REST API Design Best Practices for Parameter and Query String Usage | Moesif Blog*. Available at:

https://www.moesif.com/blog/technical/api-design/REST-API-Design-Best-Practices-for-Parameters-and-Query-String-Usage/.

Tobin, D. (no date) *Choosing the Right API: REST vs. RESTful for Integration*, *Integrate.io*. Available at: https://www.integrate.io/blog/choosing-the-right-api-rest-vs-resftul-for-integration/.